Network Working Group
Request for Comments: 0001
Category: Experimental

CSNETWK
De La Salle University - Manila
A. F. B. Laguna
April 2026

Magic: The Gathering Multiplayer Network Protocol

Version 1.0 (MTGNP)

Abstract

This document specifies the Magic: The Gathering Multiplayer Network Protocol (MTGNP), version 1.0. MTGNP defines a TCP-based, message-oriented, client-server protocol for conducting two-player, simplified Magic: The Gathering card game sessions over a network. The protocol addresses game state synchronization, turn management, priority arbitration, stack resolution, and combat mechanics. It is intended as an educational reference for implementing networked card game systems.

1. Introduction

Magic: The Gathering (MTG) is a complex collectible card game with intricate rules governing simultaneous player decisions, ordered action queues (the stack), and hidden game state. Implementing MTG over a network presents unique protocol challenges not found in simpler multiplayer games: priority windows allow both players to act at nearly every point in a game turn, and game state must be kept synchronized across clients while preserving hidden information such as each player's hand.

MTGNP defines how a central server (the Game Server) mediates between two clients. The server is the sole source of truth for all game state and validates every player action. Clients are intentionally thin: they render state received from the server and transmit player actions, but they never compute authoritative game outcomes.

This document specifies a simplified subset of the full MTG rules. Specifically, the following limitations apply to MTGNP 1.0:

- Exactly two players per game.
- Decks of between 1 and 50 cards each, drawn from a fixed, pre-defined card set. Both players may use different deck sizes.
- No replacement effects.
- No planeswalker permanents.
- No match structure (no best-of-three). After GAME_OVER, both players may immediately start a new game on the same TCP connection by sending fresh PLAYER_READY PDUs.

Future revisions may relax these limitations.

NOTE: MTGNP 1.0 does not define a card data transfer mechanism. Card costs, effects, power, toughness, and ability text are assumed to be pre-loaded by both the server and all clients from a shared out-of-band card catalog (e.g., a static JSON file distributed with the implementation). The card IDs exchanged in PDUs are keys into this shared catalog.

2. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 [RFC2119].

3. Terminology

The following terms are used throughout this document:

Active Player (AP): The player whose turn it currently is.

Non-Active Player (NAP): The player who is not currently taking their turn.

Priority: The right to take a game action. Only the player who holds priority may cast spells or activate non-mana abilities.

The Stack: A last-in, first-out (LIFO) zone where spells and abilities wait before resolving. Both players may add items to the stack whenever they hold priority.

Sequence Number (seq_num): A monotonically increasing integer present in every PDU. For server-to-client PDUs, the server increments this counter with each PDU it sends. For client-to-server action PDUs, the client MUST echo the seq_num from the most recently received PRIORITY_GRANT or corresponding server request PDU; the server MUST reject mismatches with ERROR code STALE_ACTION.

Game State: The complete authoritative set of all game information: all zones (library, hand, battlefield, graveyard, stack), life totals, turn number, and current phase.

Visible State: The subset of Game State visible to a specific player. Each player's hand is hidden from the opponent; all other zones are public.

Phase: A major division of a turn (e.g., Main Phase, Combat Phase).

Step: A subdivision of a phase (e.g., Declare Attackers Step, Declare Blockers Step).

Summoning Sickness: A creature that entered the battlefield under a player's control this turn MUST NOT be declared as an attacker and MUST NOT activate abilities with the tap symbol in their cost, unless the creature has Haste. The server enforces this rule automatically.

PDU: Protocol Data Unit. A single MTGNP message exchanged between client and server.

4. System Architecture

4.1. Client-Server Model

MTGNP uses a centralized client-server model. One process acts as the Game Server; exactly two processes act as Player Clients.

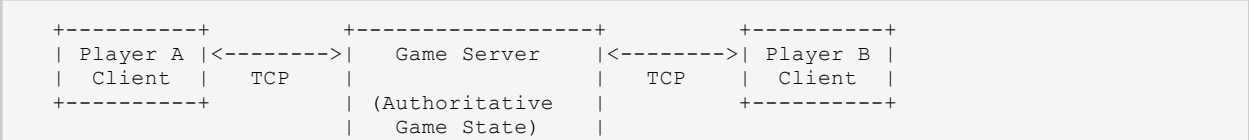

Figure 1: MTGNP Client-Server Architecture

4.2. Server Responsibilities

The Game Server MUST:

- Maintain the single authoritative copy of the Game State.
- Validate all PDUs received from clients and reject illegal actions with an ERROR message.
- Manage all phase and step transitions and broadcast PHASE_TRANSITION messages.
- Issue PRIORITY_GRANT messages to the appropriate player at the start of each priority window.
- Manage the Stack, resolving the top item when both players pass priority consecutively.
- Compute and apply all combat damage.
- Detect win/loss conditions and issue GAME_OVER messages.
- Enforce the time_limit_ms advertised in each PRIORITY_GRANT. If the priority-holding client does not respond before the deadline, the server MUST broadcast GAME_OVER with reason DISCONNECT, retain the TCP connection for the non-timed-out player, and return to LOBBY state.
- Send personalized GAME_STATE_UPDATE messages to each client, filtering out hidden information.

4.3. Client Responsibilities

A Player Client MUST:

- Maintain a local rendering of the Visible State for its player.
- Include the current seq_num in all action PDUs.
- Accept GAME_STATE_UPDATE messages from the server as the authoritative state and discard any locally computed state that conflicts.
- Send PING messages at regular intervals (RECOMMENDED: every 30 seconds) and disconnect if no PONG is received within an implementation-defined timeout (RECOMMENDED: 10 seconds after sending a PING with no response).

NOTE: Clients MUST NOT compute game outcomes locally. All game logic resides on the server. A client that attempts to validate actions locally risks displaying inconsistent state.

5. Transport and Message Framing

5.1. TCP Connection

MTGNP operates over TCP [RFC9293]. The default server port is 4444. Clients MUST initiate the TCP connection to the server. The server MUST accept connections from exactly two clients before beginning the game setup sequence. Additional connection attempts after two players are seated MUST be refused. Because TCP guarantees in-order delivery, MTGNP does not define any PDU reordering or deduplication mechanism beyond the seq_num field.

5.2. Message Framing

All PDUs are framed with a 4-byte, big-endian unsigned integer length prefix indicating the byte length of the JSON payload that follows. Receivers MUST read exactly that many bytes before attempting JSON parsing. A PDU MUST NOT exceed 65,535 bytes.

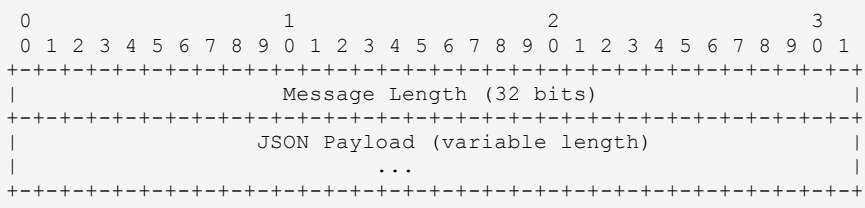

Figure 2: MTGNP Message Frame

5.3. Message Encoding

All PDUs are encoded as JSON objects [RFC8259]. All JSON MUST be valid UTF-8. Field names are case-sensitive and MUST use the exact names specified in Section 10 of this document.

5.4. General Message Structure

Every MTGNP PDU is a JSON object. Two fields MUST appear in every PDU, regardless of message type:

```
{
  "type":    "<MESSAGE_TYPE>",  // REQUIRED: string identifier for this PDU
  "seq_num": <integer>,         // REQUIRED in every PDU
  // ... additional message-specific fields ...
}
```

type: A string literal that identifies the PDU kind. The receiver MUST inspect this field first to determine how to parse the remaining fields. The complete enumeration of valid type values is given in Section 10.

seq_num: A monotonically increasing integer. For priority-bearing client-to-server PDUs (CAST_SPELL, ACTIVATE_ABILITY, PRIORITY_PASS, DECLARE_ATTACKERS, DECLARE_BLOCKERS, ASSIGN_DAMAGE_ORDER, PLAY_LAND, MULLIGAN_CHOICE, DISCARD, TRIGGER_ORDER_RESPONSE, TRIGGER_CHOICE_RESPONSE), seq_num MUST equal the value from the most recently received PRIORITY_GRANT or the corresponding server request PDU. For MULLIGAN_CHOICE specifically, the corresponding server request PDU is the GAME_STATE_UPDATE sent by the server at the start of the MULLIGAN phase (or after a redraw); the client MUST echo that PDU's seq_num. For DISCARD specifically, the corresponding server request PDU is the GAME_STATE_UPDATE the server sends at Cleanup when the hand size exceeds seven; the client MUST echo that PDU's seq_num. For DECLARE_ATTACKERS, DECLARE_BLOCKERS, and ASSIGN_DAMAGE_ORDER, the corresponding server request PDU is the PHASE_TRANSITION that signals each respective step; the client MUST echo that PHASE_TRANSITION's seq_num. The server MUST reject any such PDU whose seq_num does not match the current priority token and MUST respond with ERROR code STALE_ACTION. For server-issued PDUs, seq_num is the server's own monotonically increasing counter; receivers MAY use it for message ordering and duplicate detection. A simple counter that increments with each PDU sent is sufficient — seq_num is not required to be globally unique across the full game session.

Two client PDUs are exempt from the priority-echo rule. CONCEDE MAY be sent at any time regardless of which player holds priority; its seq_num MUST be the value from the most recently received server PDU of any type (not necessarily a PRIORITY_GRANT). PING is a heartbeat PDU whose seq_num is a client-maintained counter independent of the priority token; the server echoes it unchanged in PONG for round-trip correlation and does not validate it against the current priority token.

```
-- Player 1 acts on a stale priority grant (seq_num=14, current is 16) --

C->S    { "type": "CAST_SPELL", "seq_num": 14,
          "card_id": "counterspell_001", "targets": ["stk_02"],
          "mana_payment": { "U": 2 } }

S->P1   { "type": "ERROR", "seq_num": 15, "code": "STALE_ACTION",
          "message": "Priority token mismatch. Expected seq_num 16, got 14.",
          "rejected_action": { "type": "CAST_SPELL", "seq_num": 14 } }

-- Server re-issues the current PRIORITY_GRANT so P1 can try again --
S->P1   { "type": "PRIORITY_GRANT", "seq_num": 16,
          "player_id": "player_1", "time_limit_ms": 60000 }
```

6. Game Lifecycle

6.1. Overview

From the server's perspective, a game progresses through five top-level states:

```
  LOBBY --> GAME_SETUP --> MULLIGAN --> IN_GAME --> GAME_OVER
    ^                                                  |
    +--------------------------------------------------+
                   (loop: both players reconnect or
                    server awaits new PLAYER_READY PDUs)

              Figure 3: Game Lifecycle State Machine
```

The server MUST process these states in the order shown. After broadcasting GAME_OVER, the server MUST return to the LOBBY state and await new PLAYER_READY PDUs on the same TCP connections, allowing the same two players to start a new game without reconnecting. Any state MUST transition immediately to GAME_OVER if a player disconnects and fails to reconnect within the implementation-defined timeout.

6.2. LOBBY State

The server enters the LOBBY state upon startup and also re-enters it after every GAME_OVER. In LOBBY, the server awaits a fresh PLAYER_READY PDU from each connected player. TCP connections established at startup are reused for subsequent games; the server MUST NOT close connections at GAME_OVER unless a TCP-level error or heartbeat timeout has occurred. The server responds to each PLAYER_READY with a GAME_STATE_UPDATE reflecting the current lobby status.

The player_id field in PLAYER_READY is client-chosen and MUST be a non-empty string. The server MUST reject a PLAYER_READY whose player_id is already claimed by the other connected player, responding with ERROR code DUPLICATE_ID. Player IDs are reset at the start of each LOBBY state, so the same ID MAY be reused across games.

PLAYER_READY is exempt from the priority-echo seq_num rule. Its seq_num MUST be a client-maintained counter starting at 1 and incrementing with each PLAYER_READY sent. The server does not validate the PLAYER_READY seq_num against any priority token; it MAY use it solely for duplicate-detection or logging. A player MAY send a subsequent PLAYER_READY in the LOBBY state before both players are ready; the server MUST replace the earlier submission with the new deck list and respond with an updated GAME_STATE_UPDATE.

```
-- Player 1 connects and declares their deck (8 cards shown, up to 50 allowed) --

C->S    { "type": "PLAYER_READY", "seq_num": 1, "player_id": "player_1",
          "deck_list": [
            "lightning_bolt_001", "lightning_bolt_002", "lightning_bolt_003",
            "shock_001", "shock_002", "goblin_guide_001",
            "mountain_001", "mountain_002"
```

```
            // ... up to 50 cards total
          ] }

S->P1   { "type": "GAME_STATE_UPDATE", "seq_num": 2, "state": {
            "phase": "LOBBY", "players_ready": 1, "waiting_for": ["player_2"]
          } }

-- Invalid deck: too many cards --
C->S    { "type": "PLAYER_READY", "seq_num": 1, "player_id": "player_1",
          "deck_list": [ /* 51 cards */ ] }
S->P1   { "type": "ERROR", "seq_num": 1, "code": "ILLEGAL_DECK",
          "message": "Deck contains 51 cards; maximum is 50." }
```

Transition: When both players have sent a valid PLAYER_READY PDU, the server transitions to GAME_SETUP.

6.3. GAME_SETUP State

The server performs the following operations automatically, without requiring player input:

1. Validate both deck lists (1 to 50 cards; legal cards from the fixed set only). The server MUST reject a PLAYER_READY PDU whose deck_list is empty or contains more than 50 entries, responding with ERROR code ILLEGAL_DECK.
2. Initialize each player's life total to 20.
3. Shuffle each player's deck using a server-side random number generator.
4. Draw seven cards for each player.
5. Determine which player goes first via a random coin flip.
6. Broadcast a personalized GAME_STATE_UPDATE to each player containing the initial life totals, hand, and library count.

```
-- Server broadcasts initial state after setup (shown for Player 1) --

S->P1   { "type": "GAME_STATE_UPDATE", "seq_num": 3, "state": {
          "turn": 0, "phase": "MULLIGAN", "active_player": "player_1",
          "life_totals":     { "player_1": 20, "player_2": 20 },
          "hand":            ["lightning_bolt_001", "shock_001", "mountain_001",
                              "mountain_002", "goblin_guide_001",
                              "lightning_bolt_002", "mountain_003"],
          "hand_counts":     { "player_2": 7 },
          "library_counts":  { "player_1": 43, "player_2": 43 },
          "battlefield":     { "player_1": [], "player_2": [] },
          "graveyard":       { "player_1": [], "player_2": [] },
          "stack":           []
        } }
```

NOTE: Life totals MUST be set to 20 before the initial GAME_STATE_UPDATE is broadcast. The server MUST NOT begin the first turn until both players have completed the mulligan phase.

Transition: Immediately after setup completes, the server transitions to MULLIGAN.

6.4. MULLIGAN State

Each player independently decides whether to keep their opening hand or take a mulligan. MTGNP uses the London Mulligan rule: a player who mulligans draws a new hand of seven cards, then puts a number of cards on the bottom of their library equal to the number of times they have mulliganed.

Each player MUST send a MULLIGAN_CHOICE PDU. If keep is false, the server redraws and sends a new GAME_STATE_UPDATE to that player. A player MAY mulligan multiple times with no protocol-imposed minimum hand size. When keep is true and the player has mulliganed N times, the cards_to_bottom array MUST contain exactly N card IDs from the player's current hand. The server MUST validate this count and MUST respond with ERROR code ILLEGAL_ACTION if the array length does not match the mulligan count or contains cards not in the player's hand.

```
-- Player keeps their opening hand (seq_num 3 echoes the setup GAME_STATE_UPDATE) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": true, "cards_to_bottom": [] }

-- Player mulligans instead (seq_num 3 echoes the same setup GAME_STATE_UPDATE) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": false, "cards_to_bottom": [] }
S->C    { "type": "GAME_STATE_UPDATE", "seq_num": 4, ... }  // new 7-card hand sent

-- Player keeps after mulligan (echoes redraw GAME_STATE_UPDATE seq_num; must bottom 1
card) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 4, "keep": true,
          "cards_to_bottom": ["lightning_bolt_002"] }
```

Transition: When both players have sent MULLIGAN_CHOICE with keep: true, the server transitions to IN_GAME and begins the first player's turn.

6.5. IN_GAME State

The IN_GAME state encompasses the full turn loop described in Sections 7 through 9. The server cycles through turns, alternating the Active Player, until a win/loss condition is detected. The turn counter MUST be set to 1 when IN_GAME begins (the first player's first turn is turn 1). The server increments the turn counter at the end of each Cleanup Step before beginning the next player's Untap Step. Win conditions that MUST trigger an immediate transition to GAME_OVER are:

- A player's life total reaches zero or below.
- A player is required to draw a card from an empty library.
- A player sends a CONCEDE PDU.
- A player's connection is lost and the reconnect timer expires.

6.6. GAME_OVER State

The server broadcasts a GAME_OVER PDU to all connected players, then immediately transitions back to LOBBY state. The existing TCP connections are retained. Both players MUST send a fresh PLAYER_READY PDU to begin a new game. The reason field in GAME_OVER MUST be one of: LIFE_ZERO, DECK_EMPTY, CONCEDE, or DISCONNECT. In all cases, winner_id MUST be set to the surviving or non-offending player: for LIFE_ZERO and DECK_EMPTY the winner is the player who did

not trigger the condition; for CONCEDE the winner is the player who did not concede; for DISCONNECT the winner is the player who remained connected. If a player disconnects at this point and fails to reconnect within the implementation-defined timeout, the server MAY close that connection and re-enter a waiting state.

7. Turn Structure

7.1. Overview

Each turn consists of a fixed sequence of phases and steps. Steps that open a priority window are marked below. Unmarked steps transition automatically without player input.

```
UNTAP STEP
   |
UPKEEP STEP           <-- priority window
   |
DRAW STEP             <-- priority window
   |
PRECOMBAT MAIN PHASE  <-- priority window (sorcery speed for AP)
   |
COMBAT PHASE          <-- see Section 9 for sub-steps
   |
POSTCOMBAT MAIN PHASE <-- priority window (sorcery speed for AP)
   |
END STEP              <-- priority window
   |
CLEANUP STEP
```

Figure 4: Turn Phase Sequence

7.2. Untap Step

At the start of each turn, the server broadcasts PHASE_TRANSITION with to_phase: "UNTAP". The server then untaps all permanents controlled by the Active Player and resets land_played_this_turn to false for the new Active Player. The server broadcasts a GAME_STATE_UPDATE to both players reflecting the updated tapped state. No priority is given to either player during this step. The server MUST then broadcast PHASE_TRANSITION with to_phase: "UPKEEP" and transition immediately.

7.3. Upkeep Step

The server broadcasts PHASE_TRANSITION with to_phase: "UPKEEP" and opens a priority window with the Active Player receiving priority first. Both players may cast instants and activate abilities. The step ends when both players consecutively pass priority with an empty stack (see Section 8).

7.4. Draw Step

The server broadcasts PHASE_TRANSITION with to_phase: "DRAW" at the start of this step. The server then draws one card for the Active Player and sends a personalized GAME_STATE_UPDATE, followed

by a priority window. Note: on the very first turn of the game, the first player does NOT draw a card; the server still broadcasts PHASE_TRANSITION to DRAW and still opens a priority window, but no card is added to the hand.

7.5. Main Phases

There are two Main Phases: Precombat (before combat) and Postcombat (after combat). The server broadcasts PHASE_TRANSITION (to_phase: "PRECOMBAT_MAIN" or "POSTCOMBAT_MAIN" as appropriate) at the start of each. During Main Phases, the Active Player MAY cast sorceries, creatures, enchantments, and artifacts, and play one land per turn (playing a land does not use the stack and does not require priority). After a land is played, the Active Player retains priority; the server broadcasts an updated GAME_STATE_UPDATE and then re-issues PRIORITY_GRANT to the Active Player. Both players MAY cast instants at any time they hold priority.

Mana Abilities: Activating a mana ability (e.g., tapping a land or creature for mana) does not use the stack and does not require priority. In MTGNP 1.0, mana production is handled implicitly: the client declares the full mana_payment in the CAST_SPELL or ACTIVATE_ABILITY PDU, and the server deducts the corresponding mana sources from the game state in a single atomic step. No separate PDU is defined for mana ability activation. The server MUST respond with ERROR code INSUFFICIENT_MANA if the declared payment cannot be satisfied by the player's available mana sources.

7.6. Combat Phase

See Section 9 for the detailed Combat Phase sub-state machine.

7.7. End Step

The server broadcasts PHASE_TRANSITION with to_phase: "END_STEP". A priority window then opens. Both players may cast instants and activate abilities. The step ends when both players consecutively pass priority with an empty stack.

7.8. Cleanup Step

The server broadcasts PHASE_TRANSITION with to_phase: "CLEANUP" at the start of this step. The server first checks whether the Active Player holds more than seven cards. If so, the server MUST send a GAME_STATE_UPDATE to the Active Player reflecting the current hand, then await a DISCARD PDU listing the cards to discard until hand size is seven or fewer. A DISCARD PDU whose card_ids contains a card not in the Active Player's hand MUST be rejected with ERROR code ILLEGAL_ACTION. After each valid DISCARD, the server broadcasts an updated GAME_STATE_UPDATE reflecting the reduced hand; if the hand still exceeds seven cards, the server awaits another DISCARD PDU (the client echoes the seq_num of the most recently received GAME_STATE_UPDATE for each subsequent PDU). The server MUST NOT proceed until a valid DISCARD PDU brings the hand to seven or fewer cards. After discarding, the server removes all damage from creatures and clears any "until end of turn" effects, then broadcasts a GAME_STATE_UPDATE to both players reflecting the cleared state. No priority is given (in

MTGNP 1.0, no triggers fire at cleanup). The server increments the turn counter, switches the Active Player, and immediately begins the next turn's Untap Step.

8. Priority and the Stack

8.1. Priority Rules

The following rules govern priority in MTGNP:

1. At the start of each step that grants a priority window, the Active Player receives priority first.
2. A player who holds priority MAY cast a spell, activate a non-mana ability, or pass priority to the other player.
3. When a player casts a spell or activates an ability, the item is placed on the Stack and that player retains priority.
4. When a player passes priority, the opposing player receives priority.
5. When both players pass priority consecutively with a non-empty Stack, the server resolves the top Stack item (see Section 8.4). The Active Player then receives priority again.
6. When both players pass priority consecutively with an empty Stack, the current step ends and the server transitions to the next step.

8.2. Priority State Machine

The server maintains the following internal state machine for each priority window:

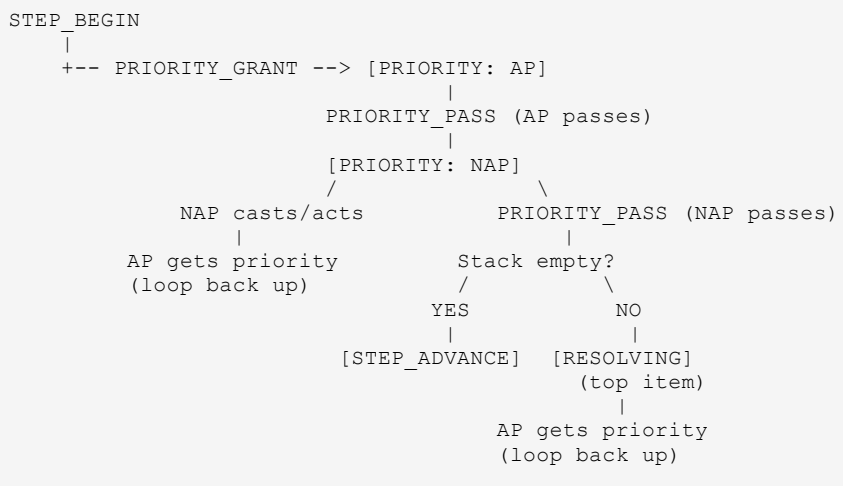

Figure 5: Priority State Machine

```
-- Full priority exchange: AP casts, NAP responds, both pass, spell resolves --

S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 5 }
C->S    { "type": "CAST_SPELL",     "seq_num": 5, "card_id": "shock_001",
          "targets": ["player_2"], "mana_payment": { "R": 1 } }
S->ALL  { "type": "STACK_PUSH", "seq_num": 6, "stack_item_id": "stk_01", ... }
```

```
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 6 }

C->S    { "type": "PRIORITY_PASS", "seq_num": 6 }  // AP passes
S->P2   { "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 7 }

C->S    { "type": "PRIORITY_PASS", "seq_num": 7 }  // NAP passes
// Both passed; stack non-empty -> RESOLVE stk_01
S->ALL  { "type": "STACK_RESOLVE", "seq_num": 8, "stack_item_id": "stk_01", "result":
"RESOLVED", ... }
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 9 }

C->S    { "type": "PRIORITY_PASS", "seq_num": 9 }  // AP passes (stack empty)
S->P2   { "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 10 }
C->S    { "type": "PRIORITY_PASS", "seq_num": 10 }  // NAP passes (stack empty)
// Both passed with empty stack -> STEP ADVANCE
S->ALL  { "type": "PHASE_TRANSITION", "seq_num": 11, "from_phase": "UPKEEP", "to_phase":
"DRAW", ... }
```

8.3. The Stack

The Stack is a LIFO data structure maintained exclusively by the server. Each Stack item contains:

- stack_item_id: A server-assigned unique identifier.
- item_type: One of SPELL, ABILITY, or TRIGGER_ABILITY.
- source_id: The card or permanent that generated this item.
- controller_id: The player who cast or activated this item.
- targets: A list of target IDs (player IDs or permanent IDs).

The server broadcasts a STACK_PUSH PDU to both players whenever an item is added to the Stack, and a STACK_RESOLVE PDU whenever an item is resolved or fizzled. In the stack array of GAME_STATE_UPDATE, index 0 represents the bottom of the Stack (the oldest item, which resolves last); the final element represents the top of the Stack (the most recently added item, which resolves first).

```
-- Player 1 (AP) casts Lightning Bolt targeting Player 2 --

C->S    { "type": "CAST_SPELL", "seq_num": 7,
          "card_id": "lightning_bolt_001", "targets": ["player_2"],
          "mana_payment": { "R": 1 } }

S->ALL  { "type": "STACK_PUSH", "seq_num": 8, "stack_item_id": "stk_01",
          "item_type": "SPELL", "source": "lightning_bolt_001",
          "targets": ["player_2"], "controller": "player_1" }

// Stack now: [ stk_01 ]  — AP retains priority
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 8, "time_limit_ms": 60000 }
```

8.4. Stack Resolution

After every game event — including spell resolution, ability activation, land play, and phase transitions — the server MUST check for state-based actions (SBAs) before granting priority to any player. SBAs that MUST be checked include: a player whose life total is zero or less loses the game (GAME_OVER with reason LIFE_ZERO); a creature with toughness zero or less is moved to its owner's graveyard; a creature

with damage marked on it equal to or greater than its toughness is destroyed and moved to the graveyard. SBAs are applied repeatedly until none remain, then triggers from those events are placed on the Stack before priority is granted. If both players' life totals reach zero or less simultaneously (for example, from mutual combat damage), the Active Player loses and the Non-Active Player wins; the server broadcasts GAME_OVER with winner_id set to the Non-Active Player.

When both players consecutively pass priority with a non-empty Stack:

1. The server pops the top item from the Stack.
2. The server checks whether all targets are still legal. If all targets are illegal, the item fizzles with no effect; the server broadcasts STACK_RESOLVE with result: FIZZLE.
3. If targets are legal, the server applies the effect, broadcasts STACK_RESOLVE with result: RESOLVED and a state_changes array describing the effect, and then broadcasts GAME_STATE_UPDATE to both players reflecting the new game state.
4. The server grants priority to the Active Player. Steps 1-4 repeat until the Stack is empty.

```
-- Lightning Bolt resolves; server broadcasts result then re-grants priority --

S->ALL  { "type": "STACK_RESOLVE", "seq_num": 11, "stack_item_id": "stk_01",
          "result": "RESOLVED",
          "state_changes": [
            { "type": "DAMAGE", "target": "player_2", "amount": 3 }
          ] }

S->ALL  { "type": "GAME_STATE_UPDATE", "seq_num": 12, ... }  // updated life totals
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 12, "time_limit_ms": 60000 }

-- If a target becomes illegal before resolution, the spell fizzles --
S->ALL  { "type": "STACK_RESOLVE", "seq_num": 13, "stack_item_id": "stk_03",
          "result": "FIZZLE", "state_changes": [] }
```

8.5. Sequence Numbers

The seq_num field is defined in Section 5.4, which provides the normative description of its semantics for both client-to-server and server-to-client PDUs. In the context of priority, the server increments its seq_num counter with each PDU it sends. A client MUST echo this value in every action PDU submitted during that priority window. Actions carrying a mismatched seq_num are rejected with ERROR code STALE_ACTION.

8.6. Triggered Abilities

Triggered abilities are game actions that fire automatically in response to specific game events. They are identified by the keywords "When", "Whenever", or "At" at the start of their text. Once triggered, they are placed on the Stack and resolve like any other Stack item. Both players may respond to triggered abilities with instants and other abilities.

8.6.1. Trigger Detection

After every game event that may cause trigger conditions to be met, the server MUST check all triggered abilities on all permanents currently on the battlefield. Events that MUST trigger a check include, but are not limited to:

- A permanent enters the battlefield.
- A permanent leaves the battlefield (destroyed, exiled, bounced, or sacrificed).
- A creature dies (is moved to the graveyard from the battlefield).
- A spell or ability is cast.
- A player draws a card.
- A step or phase begins (e.g., "At the beginning of upkeep...").
- Combat damage is dealt.

If one or more triggered abilities fire as a result of a single event, the server MUST place all of them onto the Stack before granting priority to either player. Priority MUST NOT be granted until all pending trigger ordering decisions and optional trigger choices have been resolved (see Sections 8.6.2 and 8.6.3).

8.6.2. Trigger Ordering

When multiple triggered abilities fire simultaneously, the server places them on the Stack according to the following rules:

1. All triggers controlled by the Active Player are placed on the Stack first (and thus resolve last, being at the bottom).
2. All triggers controlled by the Non-Active Player are placed on top (and resolve first).
3. If a single player controls two or more triggers that fired simultaneously, the server MUST send a TRIGGER_ORDER PDU to that player requesting the placement order. The player MUST respond with a TRIGGER_ORDER_RESPONSE PDU listing the trigger IDs in their preferred order before the Stack is updated.

```
-- Two triggers fire simultaneously for Player 1; server asks for order --

S->P1   { "type": "TRIGGER_ORDER", "seq_num": 15, "player_id": "player_1",
          "trigger_ids": ["trg_03", "trg_04"] }

-- Player wants trg_04 on stack first (so trg_03 resolves first) --
C->S    { "type": "TRIGGER_ORDER_RESPONSE", "seq_num": 15,
          "ordered_trigger_ids": ["trg_04", "trg_03"] }

S->ALL  { "type": "STACK_PUSH", "seq_num": 16, "stack_item_id": "stk_06", ... } // trg_04
S->ALL  { "type": "STACK_PUSH", "seq_num": 17, "stack_item_id": "stk_07", ... } // trg_03
(on top)
```

NOTE: TRIGGER_ORDER does not consume the player's priority — it is a mandatory ordering decision that is resolved before the Stack is updated and before any PRIORITY_GRANT is issued.

8.6.3. Optional Triggers

Some triggered abilities use the phrasing "you may", giving the controlling player a choice of whether to put the ability on the Stack. When such a trigger fires, the server MUST send a TRIGGER_CHOICE PDU

to the controlling player. The player responds with TRIGGER_CHOICE_RESPONSE containing an accept boolean. If accept is false, the trigger is discarded with no effect and no STACK_PUSH is broadcast.

```
-- A 'you may' triggered ability fires; server asks the controller --

S->P1   { "type": "TRIGGER_CHOICE", "seq_num": 20,
          "trigger_id":       "trg_02",
          "source_id":        "gray_merchant_001",
          "effect_summary":   "You may gain life equal to your devotion to black.",
          "targets":          []
        }

-- Player accepts --
C->S    { "type": "TRIGGER_CHOICE_RESPONSE", "seq_num": 20, "trigger_id": "trg_02",
          "accept": true, "chosen_target": null }

S->ALL  { "type": "STACK_PUSH", "seq_num": 21, "stack_item_id": "stk_05",
          "item_type": "TRIGGER_ABILITY", "source": "gray_merchant_001",
          "targets": [], "controller": "player_1" }

-- Player declines --
C->S    { "type": "TRIGGER_CHOICE_RESPONSE", "seq_num": 20, "trigger_id": "trg_02",
          "accept": false }
// no STACK_PUSH broadcast; trigger is silently discarded
```

Like TRIGGER_ORDER, TRIGGER_CHOICE resolution is mandatory and MUST be completed before priority is granted after any game event.

8.6.4. Triggered Abilities and the Stack

Once placed on the Stack, triggered abilities behave identically to spells: they may be responded to and they resolve from the top of the Stack downward. The server broadcasts a STACK_PUSH PDU for each triggered ability placed on the Stack, with type: TRIGGER_ABILITY, including the source permanent ID and any targets chosen at trigger resolution time.

When a triggered ability that requires a target fires, the server MUST send a TRIGGER_CHOICE PDU to the controlling player asking them to choose a legal target before the STACK_PUSH is broadcast. If no legal targets exist, the trigger is discarded immediately with no effect.

When a triggered ability resolves, the server applies its effect, broadcasts STACK_RESOLVE, and grants priority to the Active Player, exactly as described in Section 8.4.

9. Combat Phase

9.1. Overview

The Combat Phase is a sub-state machine within IN_GAME. It consists of up to six steps, each with its own priority window.

```
BEGIN_COMBAT
      |
DECLARE_ATTACKERS    <-- AP declares; priority window follows
      |
DECLARE_BLOCKERS     <-- NAP assigns blockers; priority window follows
      |
ASSIGN_DAMAGE_ORDER  <-- AP orders multi-blockers; priority window
      |
[FIRST_STRIKE_DAMAGE]<-- OPTIONAL: only if first/double strike present
      |
COMBAT_DAMAGE        <-- server resolves damage; priority window
      |
END_OF_COMBAT        <-- priority window; combat concludes
```

Figure 6: Combat Phase Sub-State Machine

9.2. Beginning of Combat Step

The server broadcasts PHASE_TRANSITION with to_phase: "BEGIN_COMBAT" and opens a priority window. This is the last opportunity for either player to act before attackers are declared. Transition occurs after both players pass with an empty stack.

9.3. Declare Attackers Step

After the priority window in the Beginning of Combat Step closes, the server broadcasts a PHASE_TRANSITION to DECLARE_ATTACKERS. This transition implicitly signals the Active Player to send a DECLARE_ATTACKERS PDU; no separate request PDU is defined. The Active Player lists all creatures they wish to attack with and their respective targets (the opposing player). An empty attackers array is legal and means no attack.

If no attackers are declared, the server MUST skip directly to the End of Combat Step. Tapped creatures and creatures with summoning sickness MUST NOT be declared as attackers; the server MUST validate and reject violations with ERROR code ILLEGAL_ACTION. Declaring a creature as an attacker taps it immediately; the GAME_STATE_UPDATE broadcast after a valid DECLARE_ATTACKERS PDU MUST reflect the updated tapped state of each declared attacker.

After a valid declaration, the server broadcasts GAME_STATE_UPDATE and opens a priority window.

9.4. Declare Blockers Step

After the priority window following attacker declaration closes, the server broadcasts a PHASE_TRANSITION to DECLARE_BLOCKERS. This transition implicitly signals the Non-Active Player to send a DECLARE_BLOCKERS PDU; no separate request PDU is defined. The Non-Active Player lists which untapped creatures block which attacking creatures. A single creature may block only one attacker; multiple creatures may block the same attacker. Blocking does not cause blocking creatures to tap; their tapped state is unchanged by the act of blocking.

After a valid declaration, the server broadcasts GAME_STATE_UPDATE and opens a priority window.

```
-- AP declares two attackers (seq_num=22 from prior PRIORITY_GRANT) --

C->S    { "type": "DECLARE_ATTACKERS", "seq_num": 22,
          "attackers": [
            { "creature_id": "goblin_guide_001", "target": "player_2" },
            { "creature_id": "reckless_wurm_003","target": "player_2" }
          ] }

S->ALL  { "type": "GAME_STATE_UPDATE", "seq_num": 23, ... }   // updated battlefield state
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 23, "time_limit_ms": 60000 }
```

9.5. Assign Damage Order Step

After the priority window following blocker declaration closes, the server broadcasts a PHASE_TRANSITION to ASSIGN_DAMAGE_ORDER if and only if at least one attacker is blocked by two or more creatures. This transition implicitly signals the Active Player to send one ASSIGN_DAMAGE_ORDER PDU per multiply-blocked attacker. The Active Player specifies the order in which each such attacker assigns its combat damage among its blockers. After all orderings have been received, the server opens a final priority window before proceeding to the damage step. If no attacker is multiply-blocked, this step is skipped and the server advances directly to the First Strike Damage Step or Combat Damage Step.

9.6. First Strike Damage Step (Optional)

This step occurs only if at least one attacking or blocking creature has first strike or double strike. The server broadcasts PHASE_TRANSITION with to_phase: "FIRST_STRIKE_DAMAGE" and then resolves first-strike damage for those creatures only. The server then checks for state-based actions (creatures with lethal damage are moved to the graveyard), broadcasts an updated GAME_STATE_UPDATE, and opens a priority window before proceeding to the regular Combat Damage Step.

9.7. Combat Damage Step

MTGNP 1.0 does not implement trample. A blocked attacker deals its full combat damage to its blocker(s) only, never to the defending player. An unblocked attacker deals damage equal to its power directly to the defending player. The server broadcasts PHASE_TRANSITION with to_phase: "COMBAT_DAMAGE" and then simultaneously assigns combat damage from all attacking and blocking creatures, excluding creatures with first strike (but NOT double strike) that already dealt damage in the First Strike Damage

Step. Double-strike creatures deal damage in both steps. The server applies all damage, updates life totals, moves creatures with lethal damage to the graveyard, and checks win conditions. It then broadcasts COMBAT_DAMAGE_RESULT, sends a personalized GAME_STATE_UPDATE to each player, and broadcasts PHASE_TRANSITION to END_OF_COMBAT; the priority window for this step is opened as described in Section 9.8.

```
-- Server resolves combat damage and broadcasts result --

S->ALL  {
          "type":           "COMBAT_DAMAGE_RESULT",
          "seq_num":        27,
          "damage_events": [
            { "source": "grizzly_bears_001", "target": "player_2",        "amount": 2 },
            { "source": "wall_of_stone_004", "target": "grizzly_bears_001","amount": 3 }
          ],
          "life_totals":    { "player_1": 20, "player_2": 18 },
          "creatures_died": ["grizzly_bears_001"]
        }

S->P1   { "type": "GAME_STATE_UPDATE", "seq_num": 28, ... }  // updated state
(personalized for P1)
S->P2   { "type": "GAME_STATE_UPDATE", "seq_num": 29, ... }  // updated state
(personalized for P2)

S->ALL  { "type": "PHASE_TRANSITION", "seq_num": 30, "from_phase": "COMBAT_DAMAGE",
          "to_phase": "END_OF_COMBAT", "active_player": "player_1" }

S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 31, "time_limit_ms": 60000 }
```

9.8. End of Combat Step

The server broadcasts PHASE_TRANSITION with to_phase: "END_OF_COMBAT" and opens a priority window. After both players pass priority with an empty stack, the server clears all combat-related state (attacker/blocker assignments, combat damage marked on permanents) and broadcasts PHASE_TRANSITION with to_phase: "POSTCOMBAT_MAIN" to begin the Postcombat Main Phase.

10. PDU Reference

10.1. PDU Summary

The following table lists all PDUs defined in this document. Direction abbreviations: C->S = Client to Server; S->C = Server to one Client; S->ALL = Server broadcast to both Clients.

Message Type	Dir	Phase	Key Fields	Notes
PLAYER_READY	C->S	Lobby	player_id, deck_list[]	1-50 cards; server rejects invalid decks with ILLEGAL_DECK
GAME_STATE_UPDATE	S->C	All	visible_state, seq_num	Personalized per player; hidden info filtered out

Message Type	Dir	Phase	Key Fields	Notes
MULLIGAN_CHOICE	C->S	Setup	keep: bool, cards_to_bottom[], seq_num	Server redraws if keep is false (London Mulligan)
PHASE_TRANSITION	S->ALL	All	from_phase, to_phase, active_player	Broadcast when server advances a step or phase
PRIORITY_GRANT	S->C	Priority	player_id, seq_num, time_limit_ms	Sent only to the player who now holds priority
PRIORITY_PASS	C->S	Priority	seq_num	seq_num must match current priority token
CAST_SPELL	C->S	Priority	card_id, targets[], mana_payment, seq_num	Server validates; pushes to stack on success
ACTIVATE_ABILITY	C->S	Priority	source_id, ability_index, targets[], seq_num	Mana abilities bypass the stack entirely
STACK_PUSH	S->ALL	Stack	stack_item_id, item_type, source, targets[]	item_type: SPELL | ABILITY | TRIGGER_ABILITY
TRIGGER_ORDER	S->C	Stack	player_id, trigger_ids[]	Player must specify order for their simultaneous triggers
TRIGGER_ORDER_RESPONSE	C->S	Stack	ordered_trigger_ids[]	Triggers listed in desired stack placement order
TRIGGER_CHOICE	S->C	Stack	trigger_id, source_id, effect_summary, legal_targets[], requires_target	Ask player to accept optional trigger or choose a target
TRIGGER_CHOICE_RESPONSE	C->S	Stack	trigger_id, accept: bool, chosen_target?	accept=false discards the trigger with no effect
STACK_RESOLVE	S->ALL	Stack	stack_item_id, result, state_changes[]	result: RESOLVED or FIZZLE
DECLARE_ATTACKERS	C->S	Combat	attackers[]: {creature_id, target}	Empty array = no attack (still required)
DECLARE_BLOCKERS	C->S	Combat	blockers[]: {creature_id, blocking_id}	Server validates legality of each block
ASSIGN_DAMAGE_ORDER	C->S	Combat	attacker_id, blocker_order[]	Required when multiple blockers on one attacker
COMBAT_DAMAGE_RESULT	S->ALL	Combat	damage_events[], life_totals, creatures_died[]	Server computes all damage simultaneously
PLAY_LAND	C->S	Main	card_id, seq_num	Does not use the stack; one per turn limit
DISCARD	C->S	Cleanup	card_ids[], seq_num	Required when hand size > 7 at cleanup
CONCEDE	C->S	Any	player_id, seq_num	Triggers immediate GAME_OVER
GAME_OVER	S->ALL	End	winner_id, loser_id, reason	reason: LIFE_ZERO | DECK_EMPTY | CONCEDE | DISCONNECT
ERROR	S->C	Any	code, message, rejected_action	Game continues; rejected action is discarded

Message Type	Dir	Phase	Key Fields	Notes
PING	C->S	Any	timestamp, seq_num	Heartbeat — server responds with PONG
PONG	S->C	Any	timestamp	Echo of the client's PING timestamp

10.2. PDU Schemas

The following subsections provide a complete JSON schema for every PDU defined in this document. Comments (// ...) are annotations only and are not part of the JSON encoding.

10.2.1. PLAYER_READY (C->S)

```
{
  "type":      "PLAYER_READY",
  "seq_num":   1,              // monotonically increasing message counter
  "player_id": "player_1",          // client-chosen non-empty string; must be unique in
this lobby
  "deck_list": [                    // 1 to 50 card IDs
    "lightning_bolt_001",
    "mountain_001",
    "goblin_guide_001"
    // ...
  ]
}
```

10.2.2. GAME_STATE_UPDATE (S->C)

GAME_STATE_UPDATE is used in two distinct contexts with different state object structures. During LOBBY, the state object contains lobby metadata. During all other phases (MULLIGAN, IN_GAME), it contains the full game state as shown below.

```
// Lobby-phase variant (sent after each PLAYER_READY):
{
  "type":    "GAME_STATE_UPDATE",
  "seq_num": 2,
  "state": {
    "phase":          "LOBBY",
    "players_ready":  1,        // how many players have sent PLAYER_READY
    "waiting_for":    ["player_2"]  // player_ids not yet ready
  }
}
```

```
// In-game variant (MULLIGAN and IN_GAME phases):
{
  "type":    "GAME_STATE_UPDATE",
  "seq_num": 44,
  "state": {
    "turn":            5,
    "active_player":   "player_1",
    "phase":           "PRECOMBAT_MAIN",
    "priority_holder": "player_1",  // null during UNTAP and CLEANUP steps
    "life_totals":     { "player_1": 17, "player_2": 12 },
    "stack": [
```

```
      { "stack_item_id": "stk_01", "item_type": "SPELL",
        "source": "lightning_bolt_001", "targets": ["player_2"],
        "controller": "player_1" }
    ],
    "battlefield": {
      // Each permanent id matches its card instance id from the original deck_list
      "player_1": [ { "id": "mountain_001", "tapped": true } ],  // Non-creatures: id and
tapped only
      "player_2": [ { "id": "wall_of_stone_004", "tapped": false,
                      "damage": 0, "power": 0, "toughness": 8,
                      "summoning_sick": false } ]  // Creatures add: damage, power,
toughness, summoning_sick
    },
    "graveyard":      { "player_1": [], "player_2": [] },  // ordered by insertion: index
0 = first card placed, last = most recently added
    "hand":           { "player_1": ["shock_002", "forest_003"] },
    "hand_counts":    { "player_2": 4 },
    "library_counts":        { "player_1": 13, "player_2": 11 },
    "land_played_this_turn": true   // true if AP has already played a land this turn
  }
}
```

10.2.3. MULLIGAN_CHOICE (C->S)

```
{
  "type":           "MULLIGAN_CHOICE",
  "seq_num":        3,                        // monotonically increasing message counter
  "keep":           true,             // false = take a mulligan
  "cards_to_bottom": ["shock_001"]    // must equal mulligan count when keep=true
}
```

10.2.4. PHASE_TRANSITION (S->ALL)

```
{
  "type":          "PHASE_TRANSITION",
  "seq_num":       10,              // server-issued sequence number
  "from_phase":    "UPKEEP",
  "to_phase":      "DRAW",
  "active_player": "player_1",
  "turn":          3
}
```

The complete set of valid string values for from_phase and to_phase, in turn order, is:

```
UNTAP            — Untap Step (no priority; server transitions immediately)
UPKEEP           — Upkeep Step
DRAW             — Draw Step
PRECOMBAT_MAIN   — Precombat Main Phase
BEGIN_COMBAT     — Beginning of Combat Step
DECLARE_ATTACKERS— Declare Attackers Step
DECLARE_BLOCKERS — Declare Blockers Step
ASSIGN_DAMAGE_ORDER — Assign Damage Order Step
FIRST_STRIKE_DAMAGE — First Strike Damage Step (optional)
COMBAT_DAMAGE    — Combat Damage Step
END_OF_COMBAT    — End of Combat Step
POSTCOMBAT_MAIN  — Postcombat Main Phase
END_STEP         — End Step
CLEANUP          — Cleanup Step
```

10.2.5. PRIORITY_GRANT (S->C)

```
{
  "type":          "PRIORITY_GRANT",
  "player_id":     "player_1",
  "seq_num":       43,
  "time_limit_ms": 60000            // server-enforced response deadline
}
```

10.2.6. PRIORITY_PASS (C->S)

```
{
  "type":    "PRIORITY_PASS",
  "seq_num": 43             // must match current PRIORITY_GRANT seq_num
}
```

10.2.7. CAST_SPELL (C->S)

```
{
  "type":         "CAST_SPELL",
  "seq_num":      7,
  "card_id":      "lightning_bolt_001",
  "targets":      ["player_2"],    // empty array if spell has no targets
  "mana_payment": { "R": 1 }       // color keys: W U B R G, generic key: "X"
}
```

10.2.8. ACTIVATE_ABILITY (C->S)

```
{
  "type":          "ACTIVATE_ABILITY",
  "seq_num":       9,
  "source_id":     "llanowar_elves_002",
  "ability_index": 0,                  // 0-based index into permanent's ability list
  "targets":       [],
  "cost_payment":  { "tap": true, "mana": {} }  // tap: true only if ability requires
tapping
  // Server rejects with ILLEGAL_ACTION if permanent is already tapped
}
```

10.2.9. STACK_PUSH (S->ALL)

```
{
  "type":          "STACK_PUSH",
  "seq_num":       8,               // server-issued sequence number
  "stack_item_id": "stk_01",
  "item_type":     "SPELL",          // SPELL | ABILITY | TRIGGER_ABILITY
  "source":        "lightning_bolt_001",
  "targets":       ["player_2"],
  "controller":    "player_1"
}
```

10.2.10. TRIGGER_ORDER (S->C)

```
{
  "type":        "TRIGGER_ORDER",
  "seq_num":     15,             // server-issued sequence number
```

```
  "player_id":   "player_1",
  "trigger_ids": ["trg_03", "trg_04"]  // player must order these
}
```

10.2.11. TRIGGER_ORDER_RESPONSE (C->S)

```
{
  "type":                "TRIGGER_ORDER_RESPONSE",
  "seq_num":             15,   // must match the corresponding TRIGGER_ORDER seq_num
  "ordered_trigger_ids": ["trg_04", "trg_03"]
  // trg_04 placed first (resolves last); trg_03 on top (resolves first)
}
```

10.2.12. TRIGGER_CHOICE (S->C)

```
{
  "type":             "TRIGGER_CHOICE",
  "seq_num":          20,           // server-issued sequence number
  "trigger_id":       "trg_02",
  "source_id":        "gray_merchant_001",
  "effect_summary":   "You may gain life equal to your devotion to black.",
  "requires_target":  false,         // true if player must also pick a target
  "legal_targets":    []             // populated when requires_target is true;
                                     // elements are player_id strings or permanent id
strings
}
```

10.2.13. TRIGGER_CHOICE_RESPONSE (C->S)

```
{
  "type":          "TRIGGER_CHOICE_RESPONSE",
  "seq_num":       20,              // must match the corresponding TRIGGER_CHOICE seq_num
  "trigger_id":    "trg_02",
  "accept":        true,
  "chosen_target": null              // non-null only when accept=true AND
requires_target=true;
                                     // absent or null when accept=false or
requires_target=false
}
```

10.2.14. STACK_RESOLVE (S->ALL)

```
{
  "type":          "STACK_RESOLVE",
  "seq_num":       31,              // server-issued sequence number
  "stack_item_id": "stk_01",
  "result":        "RESOLVED",       // RESOLVED | FIZZLE
  "state_changes": [
    { "change_type": "DAMAGE", "target": "player_2", "amount": 3 },
    { "change_type": "LIFE_GAIN", "target": "player_1", "amount": 2 },
    { "change_type": "DESTROY",   "target": "wall_of_stone_004" }
  ]
}
```

10.2.15. DECLARE_ATTACKERS (C->S)

```
{
  "type":      "DECLARE_ATTACKERS",
  "seq_num":   22,
  "attackers": [
    { "creature_id": "goblin_guide_001", "target": "player_2" },
    { "creature_id": "reckless_wurm_003","target": "player_2" }
  ]
  // send empty attackers array to declare no attack
}
```

10.2.16. DECLARE_BLOCKERS (C->S)

```
{
  "type":     "DECLARE_BLOCKERS",
  "seq_num":  24,
  "blockers": [
    { "creature_id": "wall_of_stone_004", "blocking_id": "goblin_guide_001" }
  ]
  // send empty blockers array to not block
}
```

10.2.17. ASSIGN_DAMAGE_ORDER (C->S)

```
{
  "type":          "ASSIGN_DAMAGE_ORDER",
  "seq_num":       26,
  "attacker_id":   "reckless_wurm_003",
  "blocker_order": ["wall_of_stone_004", "grizzly_bears_002"]
  // damage assigned to wall first, overflow goes to bears
}
```

10.2.18. COMBAT_DAMAGE_RESULT (S->ALL)

```
{
  "type": "COMBAT_DAMAGE_RESULT",
  "seq_num":         27,              // server-issued sequence number
  "damage_events": [
    { "source": "goblin_guide_001",  "target": "player_2",       "amount": 2 },
    { "source": "wall_of_stone_004", "target": "goblin_guide_001","amount": 3 }
  ],
  "life_totals":     { "player_1": 20, "player_2": 18 },
  "creatures_died": ["goblin_guide_001"]
}
```

10.2.19. PLAY_LAND (C->S)

```
{
  "type":     "PLAY_LAND",
  "seq_num": 5,
  "card_id": "mountain_003"
  // does not use the stack; one land play permitted per turn
}
```

10.2.20. DISCARD (C->S)

```
{
```

```
  "type":     "DISCARD",
  "seq_num":  50,
  "card_ids": ["lightning_bolt_004", "shock_003"]
  // sent at cleanup when hand size exceeds 7
}
```

10.2.21. CONCEDE (C->S)

```
{
  "type":      "CONCEDE",
  "seq_num":   99,
  "player_id": "player_2"
}
```

10.2.22. GAME_OVER (S->ALL)

```
{
  "type":      "GAME_OVER",
  "seq_num":   100,              // server-issued sequence number
  "winner_id": "player_1",
  "loser_id":  "player_2",
  "reason":    "LIFE_ZERO"
  // reason: LIFE_ZERO | DECK_EMPTY | CONCEDE | DISCONNECT
}
```

10.2.23. ERROR (S->C)

```
{
  "type":             "ERROR",
  "seq_num":          14,            // echoes the seq_num of the rejected action when
available
  "code":             "STALE_ACTION",
  "message":          "Priority token mismatch. Expected seq_num 16, got 14.",
  "rejected_action": { "type": "CAST_SPELL", "seq_num": 14, "card_id": "..." }
}
```

10.2.24. PING (C->S)

```
{
  "type":      "PING",
  "seq_num":   1,              // used to correlate with PONG response
  "timestamp": 1745000000000   // Unix epoch milliseconds
}
```

10.2.25. PONG (S->C)

```
{
  "type":      "PONG",
  "seq_num":   1,              // echoes the PING seq_num
  "timestamp": 1745000000000   // echoes the PING timestamp
}
```

11. Error Handling

When the server receives an invalid or illegal PDU from a client, it MUST:

1. Send an ERROR PDU to the originating client containing: an error code (see below), a human-readable message string, and a copy of the rejected action PDU.
2. Discard the illegal action and leave the game state unchanged.
3. If the player still holds priority, re-issue PRIORITY_GRANT with the same seq_num so the player may try again.

Defined error codes:

INVALID_JSON: The received bytes could not be parsed as valid UTF-8 JSON.

ILLEGAL_DECK: The submitted deck_list is empty, contains more than 50 cards, or includes one or more cards not in the legal card set.

UNKNOWN_TYPE: The type field does not match any known PDU type.

STALE_ACTION: The seq_num does not match the current priority token.

NOT_YOUR_PRIORITY: The client submitted an action PDU when it does not hold priority.

ILLEGAL_ACTION: The action is syntactically valid but violates game rules (e.g., attacking with a tapped creature).

ILLEGAL_TARGET: One or more targets in a CAST_SPELL, ACTIVATE_ABILITY, or TRIGGER_CHOICE_RESPONSE PDU are not legal targets.

TRIGGER_ORDER_INVALID: The TRIGGER_ORDER_RESPONSE does not contain exactly the trigger IDs that were sent in the corresponding TRIGGER_ORDER PDU.

TRIGGER_CHOICE_INVALID: The TRIGGER_CHOICE_RESPONSE references an unknown trigger_id, or chosen_target is absent when a target is required.

INSUFFICIENT_MANA: The mana_payment provided does not satisfy the spell's mana cost.

WRONG_PHASE: The action is not legal in the current phase (e.g., casting a sorcery outside a Main Phase).

DUPLICATE_ID: The player_id in a PLAYER_READY PDU is already claimed by the other connected player in this lobby session.

NOTE: The server MUST NOT disconnect a client solely because it received an illegal action PDU. Disconnection MUST only occur on TCP-level errors or a heartbeat timeout.

12. Security Considerations

This document specifies a protocol intended for educational use in a controlled local network environment. The following security considerations are noted for completeness.

Authentication: MTGNP 1.0 does not define an authentication mechanism. Deployments where player identity matters SHOULD implement an application-layer authentication handshake before PLAYER_READY is accepted.

Confidentiality: MTGNP 1.0 transmits all data, including player hand contents, as plaintext JSON over TCP. Deployments over untrusted networks SHOULD wrap TCP connections in TLS [RFC8446].

Cheating Prevention: Because all game logic resides on the server and every action is independently validated, a cheating client cannot force an illegal game state. The server MUST withhold hidden information (opponent hand contents) from GAME_STATE_UPDATE messages.

Denial of Service: A malicious client could stall the game by never sending required PDUs. Implementations SHOULD enforce the time_limit_ms field in PRIORITY_GRANT. A player who does not respond within the time limit SHOULD be treated as disconnected.

13. References

13.1. Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

[RFC8259] Bray, T., "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 8259, December 2017.

[RFC9293] Eddy, W., "Transmission Control Protocol (TCP)", RFC 9293, August 2022.

13.2. Informative References

[MTG-CR] Wizards of the Coast, "Magic: The Gathering Comprehensive Rules", current edition. https://magic.wizards.com/en/rules

[RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, August 2018.

Author's Address

A. F. B. Laguna
De La Salle University - Manila
CSNETWK — Computer Networks
Email: ann.laguna@dlsu.edu.ph